Benchmarking Post-Quantum Key Encapsulation in S3-Compatible Object Storage under Client Concurrency

Team 12 (author names to be added)
Department and institution to be added

***Abstract*—Large-scale quantum computers would break the RSA and elliptic-curve key establishment that protects most encrypted cloud data, and ciphertext stored today can be decrypted later. NIST standardized ML-KEM (FIPS 203) as the replacement key-encapsulation mechanism, but its cost has mostly been measured in isolation or inside TLS handshakes, not in the per-object encryption path of cloud object stores under concurrent load. We build an instrumented client-side envelope-encryption layer for an S3-compatible store (MinIO) in which every object is protected by ML-KEM-768, HKDF-SHA256 and chunked AES-256-GCM, and compare it on one data path with no encryption, symmetric-only encryption, RSA-OAEP-4096, X25519 and a hybrid X25519+ML-KEM-768 mode, from 1 to 500 concurrent workers and from 1 KiB to 500 MiB objects, on two separate cloud instances (1,970 runs, 101.3 million measured requests, 0 failures). ML-KEM-768 costs 0.045 ms per PUT and 0.047 ms per GET, at most 2.25 % of request time even for the smallest objects. In 38 of 44 paired comparisons its throughput is statistically equivalent to X25519 within ± 5 %, and none is confirmed to differ by more (6 are inconclusive). Every mode reaches the same network, storage or client-CPU ceiling, except RSA-4096, whose 1.486 ms decryption saturates the client and cuts small-object GET throughput to 0.38–0.59 of ML-KEM's. Bounded session amortization of ML-KEM saves 30–66 µs of client CPU per request at one worker and up to 100 µs under contention, which raises throughput only where the client is the bottleneck. Open-loop load shows a sharp knee between $\rho = 0.58$ and 0.83 that an M/M/c model does not predict.**

***Index Terms*—post-quantum cryptography, ML-KEM, object storage, S3, envelope encryption, benchmarking**

I. Introduction

Cloud object stores keep data for years. An adversary who records encrypted objects or key-exchange traffic today can decrypt them once a cryptographically relevant quantum computer exists ("harvest now, decrypt later"). The standard response is to replace RSA and elliptic-curve key establishment with post-quantum key-encapsulation mechanisms. In August 2024 NIST published FIPS 203, standardizing ML-KEM (derived from CRYSTALS-Kyber) and recommending ML-KEM-768 as the default parameter set [1].

For object storage the question is practical. Every object stored with client-side envelope encryption needs its own key establishment, a larger metadata record and more client CPU. Whether this matters depends on object size, the operation and how many clients compete for the same CPUs, network and storage servers. Prior measurements concentrate on algorithm micro-benchmarks and TLS handshakes; cloud-storage proposals evaluate security properties and single-client encryption time (§III).

We ask three questions. **RQ1:** What does per-object ML-KEM-768 add to latency, throughput, CPU and memory, compared with RSA-4096, X25519 and symmetric-only baselines on the same data path? **RQ2:** How does this change with object size and with 1–500 concurrent workers? **RQ3:** Can a queueing model predict where response time degrades, and can bounded session amortization of ML-KEM move that point?

Contributions. (i) An instrumented, reproducible benchmark of per-object ML-KEM-768 envelope encryption on an S3-compatible store with six key-establishment modes on one data path, stage-level latency decomposition, client and server resource sampling and streaming memory for objects of hundreds of MiB. (ii) A measurement on two separate cloud instances with five randomized repetitions per configuration, paired within-batch comparisons and explicit equivalence tests. (iii) An evaluation of bounded session amortization of ML-KEM, including a variant whose bound follows client utilization. Bounded key reuse itself is established practice (§III); our claim is its application to ML-KEM roots, its quantified trade-off, and the queue-aware variant as a candidate policy. (iv) The artifact, whose validator refuses data with inconsistent timers, checksums or session bounds.

II. Background

ML-KEM. A KEM has three algorithms: key generation, encapsulation (which, given the recipient's public key, outputs a ciphertext and a 32-byte shared secret) and decapsulation. ML-KEM-768 targets NIST security category 3, with a 1,184-byte encapsulation key, a 2,400-byte decapsulation key and a 1,088-byte ciphertext [1]. Decapsulation uses *implicit rejection*: a modified ciphertext or a wrong key yields a pseudo-random secret instead of an error, so tampering is detected by the authenticated encryption that follows.

Envelope encryption. A per-object data key encrypts the object; key establishment protects that data key for the recipient. We derive keys with HKDF-SHA256 [2] and encrypt with AES-256-GCM, which requires nonce uniqueness per key [3].

Object stores. S3-compatible stores expose PUT and GET of whole objects. We use MinIO [4]. Its community edition was archived in 2026 and no longer publishes images, so we compile the final community release (RELEASE.2025-

10-15T17-29-55Z) from source; because all cryptography is client-side, any S3-compatible store can be substituted.

III. Related Work

Algorithm-level cost. Library benchmarks such as those in liboqs [5] and comparative studies of ML-KEM against RSA and ECC [6] quantify key generation, encapsulation and decapsulation and the larger key and ciphertext sizes. They do not include storage, concurrency or end-to-end request latency.

Transport security. Hybrid key exchange in TLS 1.3 [7], with combiners such as X-Wing [8], protects data in transit; session resumption amortizes the handshake across connections. Our work concerns keys protecting data at rest, one per object, and holds transport security constant.

Post-quantum cloud storage. Prikutse et al. [9] combine AES-256-GCM, X25519+ML-KEM-768 and ML-DSA-65 for multi-file cloud storage and amortize signatures by batch commitment. According to its abstract, that evaluation does not cover an S3 store, concurrent clients or queueing behaviour.

Bounded key reuse. Amazon S3 Bucket Keys derive per-object data keys from a time-limited bucket-level key and reduce key-service requests by up to 99% [10]; the AWS Encryption SDK caches data keys with age and message-count limits [11]. Our session mode applies the same principle to ML-KEM-established roots in client-side encryption. In vendor documentation and arXiv we found no prior load-adaptive bound; a search of the academic databases is still to be completed.

IV. Threat Model and Design

Threat model. The network and object store are honest-but-curious for confidentiality and potentially malicious for integrity: an adversary may read, replace, delete, reorder or replay stored ciphertexts and manifests. The adversary does not hold the recipient's decapsulation key. Availability, traffic analysis and side channels in the Python client are out of scope.

Construction (mode P1). For object identity $I = (tenant, bucket, key, version)$ the client computes $(C, S) \leftarrow \mathrm{Encaps}(ek)$ and $K \leftarrow \mathrm{HKDF}(S, salt, \text{“pqc-s3-object-v2”} \| 0 \| ctx)$, where ctx is canonical JSON of I, the mode, algorithm names, chunk size, chunk count, plaintext size, recipient key identifier and a hash of C. The object is split into 4 MiB chunks; chunk j uses nonce $p \| \mathrm{be}_{64}(j)$ with a random 32-bit prefix p and associated data $ctx \| j \| \ell_j \| final_j$. Deletion, reordering, truncation, extension and cross-object splicing therefore fail authentication. The ciphertext is uploaded first and a manifest holding ctx, C, the salt and the prefix is written last as the commit record, so an interrupted upload is never visible. Object versions are time-ordered UUIDv7 values.

Comparison modes. All encrypting modes share the HKDF and chunked AES-GCM path: B0 stores plaintext through the same framing (storage and harness cost); B1 derives keys from a pre-shared 32-byte key; R1 transports a random 32-byte secret with RSA-OAEP-4096 [12]; B2 uses ephemeral-static X25519 [13]; P2 concatenates X25519 and ML-KEM-768 secrets. A reader must name the mode it expects, so a planted plaintext object cannot downgrade P1.

Bounded sessions (mode N1). One encapsulation yields a root S_r; object i uses $K_i = \mathrm{HKDF}(S_r, salt_i, \text{“pqc-s3-session-v1”} \| 0 \| ctx_i)$, with the session identifier, the object's sequence number and the bound authenticated in ctx_i. A root protects at most B objects and lives at most $T_{\max}$ seconds. In the queue-aware variant, $B_t = \mathrm{clip}(B_{\min}, B_{\max}, \mathrm{round}(B_{\min} + (B_{\max} - B_{\min})\hat{\rho}))$ with $\hat{\rho}$ an exponentially weighted estimate of the fraction of busy worker threads: under light load a root is used for fewer objects. B_t is fixed per session, so $B_{\max}$ and $T_{\max}$ are hard caps. A leaked root exposes at most $B_{\max}$ objects; compromise of the recipient's decapsulation key exposes all objects in both P1 and N1. Receivers cache decapsulated roots under their own caps and reject objects whose authenticated sequence number exceeds them.

V. Implementation

The client is Python 3.12.3 using liboqs 0.16.0 through liboqs-python 0.16.0.1 [5], [14], built from a pinned release with only ML-KEM enabled, pyca/cryptography 50.0.1 [15] for HKDF, AES-GCM, RSA and X25519, and the MinIO Python SDK 7.2.20. A test checks ML-KEM interoperability with an independent implementation (OpenSSL via pyca) in both directions. Storage is streamed: objects are encrypted chunk by chunk and uploaded in 8 MiB parts, so memory stays bounded regardless of object size. A multi-process runner realizes N concurrent workers as exactly N threads over $\min(N, 16)$ processes that start together after warm-up. It records every request, including failures, with stage timers (key establishment, key derivation, AES-GCM, manifest handling and storage), and samples client resources and MinIO's own process metrics once per second. A validator rejects a data set if checksums fail, a record deviates from the schema, any request's stage times exceed its total, a run's 99th-percentile unaccounted time exceeds 2 %, or an N1 session exceeds its bound. The test suite has over two hundred tests, including tamper, replay, downgrade and session-bound stress tests.

VI. Methodology

Testbed. Two AWS instances in one cluster placement group (ap-south-2): the client on a c7i.4xlarge (Intel Xeon Platinum 8488C, 16 vCPU, 31 GiB) and MinIO, single node in Docker with host networking, on an i4i.4xlarge (16 vCPU, local NVMe). Baselines before the runs: 12.4 Gbit/s (1,479 MiB/s) between the hosts with iperf3, a 0.33 ms round trip, and 2,044 MiB/s read and 882 MiB/s write on the MinIO disk with fio. Code at commits `4597849`, `4f6f8a1`, `ef30535`.

Workload. PUT and GET are measured separately in a closed loop (each worker issues its next request when the previous one completes), with 5 s warm-up and 20 s measured

TABLE I
BATCHES AND THE RESULTS EACH ONE SUPPLIES

Batch	Runs	Requests	Result of record for
A	850	35.4 M	concurrency sweep: GET; PUT $\geq$1 MiB; hybrid P2
B	360	1.0 M	object size 1 KiB–500 MiB; handshake cost
C	500	46.9 M	open loop (RQ3); N1 on GET; queue-aware N1
D	260	18.0 M	PUT $\leq$64 KiB incl. N1 (40 s pause)

per run and fresh worker processes for every run. Concurrency 1–500 is measured at 64 KiB and 1 MiB; 16 MiB stops at 128 workers because 500 objects of 16 MiB in flight exceed the client's memory; object size 1 KiB–500 MiB is measured at one worker, with 128 MiB at 32 and 500 MiB at 16 workers. Large objects require at least 10 measured operations per run. For RQ3, an open-loop generator offers Poisson arrivals of 1 KiB and 64 KiB PUTs, served by 256 threads, at rates around the measured capacity. N1 is measured at $B_{\max} \in \{10, 100, 1000\}$, $T_{\max} = 60$ s, and the queue-aware variant at $B_{\min} = 1$, $B_{\max} = 100$.

Replication and statistics. Every configuration runs five times; the order of modes is shuffled within each repetition. A run is the unit of analysis: we report the mean over runs with a Student-t 95 % interval. Modes are compared only within one batch, paired by repetition, as the ratio $\exp(\overline{\log(x_a/x_b)})$ with a t-interval; a comparison is *equivalent* if its whole interval lies within ± 5 %, *different* if it lies entirely outside, and *inconclusive* otherwise. Above one worker, stage timers include time spent waiting for the Python interpreter lock, so we report key-establishment cost and stage shares at one worker and client CPU as thread CPU time per request.

Batches. The study ran in four batches (Table I). In batches A–C the pause between runs, meant to wait until MinIO stopped writing, fell back to 5 s because MinIO in Docker does not export its disk-write counter. A PUT run that followed another PUT run was then sometimes slowed by 12–24 %, consistent with the previous run's writeback and object purge still in progress: 38 small-object PUT cells exceeded our 5 % limit on run-to-run variation and 17 runs were flagged as stalls. Because mode order was shuffled, this added noise but no bias. Batch D re-measured all PUT cells up to 64 KiB with a 40 s pause, covering the kernel's 30 s dirty-page writeback and MinIO's 5 s cleanup interval: the variation of P1 at 64 KiB and 128 workers fell from 11.7 % to 0.7 % (B1: 10.9 % to 0.3 %), with 3 stalls in 260 runs. Batch D is the result of record for those cells.

VII. RESULTS

Handshake overhead (RQ1). Table II gives key-establishment time per request. ML-KEM-768 costs 0.045 ms to encapsulate on PUT and 0.047 ms to decapsulate on GET. X25519 costs 1.9× more on PUT, where the sender generates an ephemeral key, and about the same on GET. RSA-4096

TABLE II
KEY ESTABLISHMENT PER REQUEST (MEAN [95 % CI], ONE WORKER, OBJECTS UP TO 16 MIB, BATCH B) AND MANIFEST SIZE

Mode	PUT (ms)	GET (ms)	Manifest (B)
B0 plaintext	–	–	522
B1 AES only	0.001 [0.001, 0.001]	0.001 [0.001, 0.001]	576
R1 RSA-4096	0.094 [0.094, 0.094]	1.486 [1.484, 1.488]	1,276
B2 X25519	0.087 [0.086, 0.088]	0.049 [0.049, 0.049]	622
P1 ML-KEM-768	0.045 [0.045, 0.046]	0.047 [0.046, 0.047]	2,034
P2 hybrid	–	–	2,091

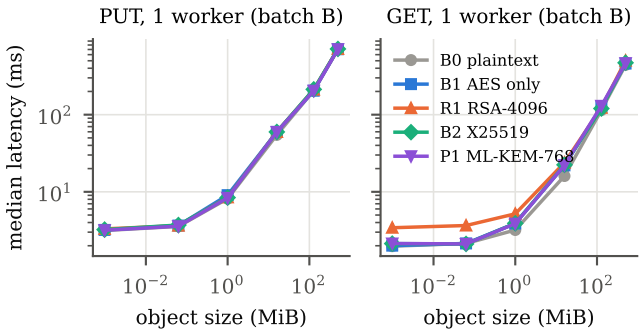


Fig. 1. Median latency against object size, one worker (batch B; log scales).

is cheap to encrypt but costs 1.486 ms to decrypt, 32× ML-KEM decapsulation, which places its cost on the read path. The price of ML-KEM is metadata: its manifest is 2,034 bytes against 622 for X25519 and 1,276 for RSA-4096, because it carries the 1,088-byte ciphertext.

Object size (RQ2). Fig. 1 shows median latency against object size at one worker. For P1 the share of request time spent in ML-KEM falls from 1.41 % (PUT) and 2.19 % (GET) at 1 KiB to 0.012 % and 0.010 % at 500 MiB; even a 1 KiB PUT takes 3.18 ms, dominated by two storage round trips (data, then the committing manifest). The only mode visibly apart is R1 on GET: its median latency is 73 % above P1 at 64 KiB and 34 % at 1 MiB.

Concurrency and throughput (RQ2). Fig. 2 shows throughput from 1 to 500 workers. All modes reach the same ceilings. GETs of 1 MiB and more saturate the network at 1,477 MiB/s, 100 % of the iperf3 baseline; 1 MiB PUTs level off at about 1,310 MiB/s. Small PUTs level off at about 4,700 requests/s (295 MiB/s at 64 KiB) while the client uses at most 49 % of its CPU, the server 72 % and the network 21 % of the link: by elimination the limit is MinIO's write path, possibly including its CPU, since 72 % of 16 vCPU leaves little headroom on 8 physical cores with hyper-threading. Small GETs are limited by the client CPU (93 % for P1 at 128 workers, 9,400 requests/s). RSA-4096 reaches that limit much earlier: at 98 % client CPU it delivers 3,600 requests/s, 0.38–0.59 of P1's throughput at 8–500 workers. For objects of 1 MiB and more, where the network or storage is the bottleneck, R1 and P1 are equivalent from 32 workers on.

ML-KEM against X25519 (H1). Table III summarizes the paired throughput ratios P1/B2 over the 44 closed-loop cells

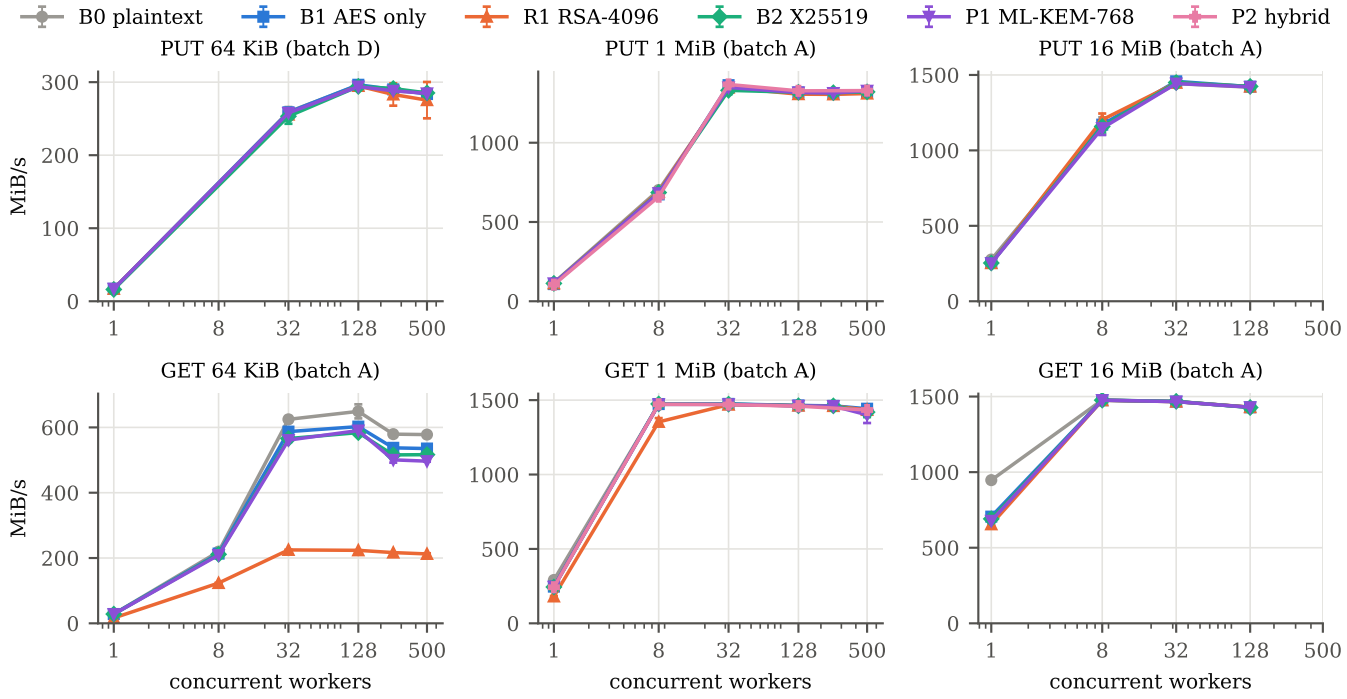


Fig. 2. Throughput (MiB/s, mean and 95 % CI over five runs) against concurrent workers. 64 KiB PUT from batch D, others from batch A. The hybrid P2 was measured at 1 MiB only.

of record. 38 are equivalent within ±5 %, 6 inconclusive and 0 different; equivalent ratios lie between 0.961 and 1.039. At the small-PUT ceiling the ratio is 0.999 [0.993, 1.006]. In 7 of the equivalent cells the interval still excludes 1, so a small difference exists inside the margin; the largest is where the client CPU is the bottleneck (64 KiB GET, 500 workers): P1 delivers 3.9 % less than B2 [0.959, 0.964]. The inconclusive cells lie at the knee of the throughput curve (8–32 workers) or at one worker, where a single slow run widens the interval.

CPU and memory (RQ1). Table IV lists CPU and memory for representative workloads. Per request, P1 uses between −0.045 and 0.048 ms more client CPU than X25519 at one worker, and 1.37–1.63 ms less than RSA-4096 on GET. Server CPU does not depend on the mode. Client memory is dominated by the interpreter and buffers: about 2,060 MiB at 500 workers, within 2.5 % across modes; its growth during a run tracks throughput (R1, the slowest on GET, grows least), and streaming keeps the growth of a single 500 MiB PUT at most 20 MiB. Server memory growth comes mostly from single runs in which MinIO grew its heap (P1 at 1 MiB PUT: one run 756 MiB, the others at most 20 MiB); its intervals overlap across modes in 4 of 4 workloads, so it does not depend on the key-establishment mode.

Hybrid mode. P2 (X25519+ML-KEM-768) is equivalent to P1 in 7 of the 8 cells at 1 MiB from 8 workers on. At one worker its PUT throughput is 7.3 % lower. Most of that gap, like the higher cost of B1 at the same size, sits in the AES-GCM stage (0.37 ms for P2 and 0.29 ms for B1 against 0.12 ms for P1), although all modes run the identical cipher code: it arises outside the cipher and we did not isolate its cause. At 1 MiB we therefore use B0 and B2, not B1, as references.

TABLE III
P1 AGAINST X25519 (B2): PAIRED THROUGHPUT RATIOS AND VERDICTS (EQUIVALENT / INCONCLUSIVE / DIFFERENT, MARGIN ±5 %) OVER THE CELLS OF RECORD

Batch	Op	Sizes	Workers	Eq./Inc./Diff.	P1/B2 throughput
A	GET	1 MiB, 16 MiB	1, 8, 32, 128, 256, 500	9/1/0	0.981–1.005
A	GET	64 KiB	1, 8, 32, 128, 256, 500	6/0/0	0.961–1.011
A	PUT	1 MiB, 16 MiB	1, 8, 32, 128, 256, 500	9/1/0	0.990–1.014
B	GET	1 MiB, 16 MiB, 128 MiB, 500 MiB	1	3/1/0	0.921–1.011
B	GET	1 KiB, 64 KiB	1	2/0/0	0.991–1.000
B	PUT	1 MiB, 16 MiB, 128 MiB, 500 MiB	1	4/0/0	1.000–1.039
D	PUT	1 KiB, 64 KiB	1, 32, 128, 256, 500	5/3/0	0.989–1.060

Bounded sessions. With a bound $B_{\max}$, N1 performed one ML-KEM operation per $B_{\max}$ objects (0.0100 per request at $B_{\max} = 100$), as designed and as the validator checked for every session. Fig. 3 shows the effect. At one worker N1

TABLE IV
CPU (% OF THE HOST'S 16 VCPU, MEASURED WINDOW), CLIENT CPU TIME PER REQUEST AND MEMORY (PEAK RSS AND GROWTH OVER THE RUN), MEAN OF FIVE RUNS

Workload	Mode	Client CPU %	Server CPU %	CPU ms/req	Client peak MiB	Client growth MiB	Server growth MiB
GET 64 KiB c128 (A)	B1	91	71	1.49	2,144	367	4
GET 64 KiB c128 (A)	R1	98	23	4.36	1,911	125	5
GET 64 KiB c128 (A)	B2	92	69	1.54	2,134	349	0
GET 64 KiB c128 (A)	P1	93	70	1.56	2,110	331	1
PUT 1 MiB c128 (A)	B1	34	53	3.92	2,283	265	16
PUT 1 MiB c128 (A)	R1	42	51	4.97	2,192	116	37
PUT 1 MiB c128 (A)	B2	35	52	4.06	2,298	126	33
PUT 1 MiB c128 (A)	P1	35	51	4.07	2,291	254	158
PUT 500 MiB c16 (B)	B1	25	47	675.42	2,698	174	70
PUT 500 MiB c16 (B)	R1	25	47	676.55	2,719	188	27
PUT 500 MiB c16 (B)	P1	26	47	680.14	2,729	202	18
GET 500 MiB c16 (B)	B1	23	6	1,033.11	2,011	12	90
GET 500 MiB c16 (B)	R1	23	6	1,024.33	2,025	28	144
GET 500 MiB c16 (B)	P1	22	6	997.48	2,011	33	67

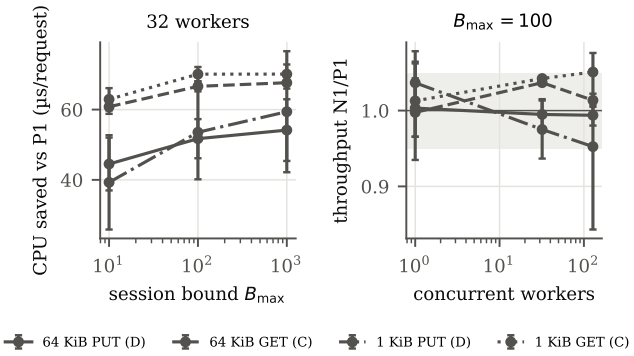


Fig. 3. N1 against P1: client CPU saved per request (left, 32 workers) and paired throughput ratio at $B_{max} = 100$ (right; shaded band ±5 %). PUT from batch D, GET from batch C.

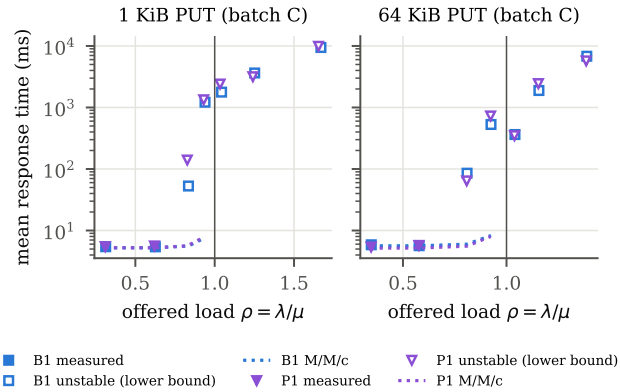


Fig. 4. Open-loop mean response time against offered load (batch C, log scale). Hollow markers: unstable runs, whose response time grows with run length (lower bounds). Dotted: M/M/c prediction.

saves 30–66 µs of client CPU per request against P1, matching the 38–42 µs of key-establishment time it removes; with 32–128 workers the saving grows to 49–100 µs, because each ML-KEM call through the Python binding costs more CPU under contention. All these intervals exclude zero. Because ML-KEM is a small part of every request, this is visible as throughput only where the client CPU is the bottleneck: small GETs at 32–128 workers (client CPU 92 %) run 1.4–5.1 % faster. On PUT, where MinIO limits throughput, N1 is equivalent to P1 in 3 of 6 cells and inconclusive in the rest. The queue-aware variant adapts as intended: under open-loop load of 1,500 and 3,500 requests/s its mean bound was 18.4 and 31.9, so it re-keyed 5.5× and 3.4× as often as the fixed bound of 100, with a 99th-percentile response-time ratio of 1.02 [0.98, 1.07] and 0.94 [0.67, 1.33]; under overload its bound rose to 100.0. This shows that the policy works at no measurable cost; it does not show that it is better than a smaller fixed bound.

Queueing (RQ3). Fig. 4 shows mean response time (queue wait plus service) against offered load $\rho = \lambda/\mu$, with capacity μ taken from the most overloaded runs (4,800 requests/s at 1 KiB, 4,300 at 64 KiB). We model the system as M/M/c [16] with $c = \mu S_0$ servers, where S_0 is the service time at light load (5.2 ms; $c = 25$ at 1 KiB, 22 at 64 KiB). Below the knee the model is within 7–10 % of the measurements. It misses the knee: at 1 KiB and 4,000 requests/s ($\rho = 0.83$) it predicts 5.6 ms while the system was already unstable, with mean response above 140 ms; at 64 KiB the same happens at $\rho = 0.81$ (predicted 5.6 ms, measured above 64 ms). The knee lies between $\rho = 0.63$ and 0.83 at 1 KiB and between 0.58 and 0.81 at 64 KiB, for B1 and P1 alike. The model fails because service time is not independent of load: the median service time of a 1 KiB PUT rises from 5.0 ms at light load to about 50 ms in overload, as concurrent writes contend inside MinIO. Usable capacity is therefore between 0.58 and 0.83 of the saturation throughput, and ML-KEM does not move it.

VIII. DISCUSSION

For per-object envelope encryption on an S3-compatible store, ML-KEM-768 is not a performance problem. Its key establishment costs tens of microseconds, less than X25519

on PUT in our stack, and disappears against the milliseconds of storage round trips; on every workload we measured, throughput is set by the network, by MinIO's write path or, for small GETs, by the client CPU, and ML-KEM reaches those limits together with the classical modes. The measurable costs of migration are elsewhere: 1,412 more bytes of metadata per object than X25519, which matters for very small objects and for object-count-based billing, and a few tens of microseconds of client CPU per request, which matters only when the client is saturated. RSA-4096, in contrast, is a poor baseline to stay on: its private-key operation makes small-object reads CPU-bound at 0.38–0.59 of the throughput of ML-KEM. Session amortization is useful exactly where the client CPU is the bottleneck; elsewhere it trades compromise scope for no measurable gain. For capacity planning, the open-loop results matter more than any cryptographic choice: response time collapses well before nominal saturation.

IX. Threats to Validity

Internal. The pause between runs failed in batches A–C (§VI); batch D repeated the affected cells with a working pause, but 8 of its 52 cells still exceed 5 % variation. Absolute small-object throughput differed by up to 13 % between batches, so all comparisons are within one batch. Stage timers above one worker include interpreter-lock waiting. Five repetitions give t-intervals with four degrees of freedom; the attribution of the small-PUT limit to MinIO's write path is by elimination, and the AES-GCM cost difference between modes at 1 MiB is unexplained. **External.** The client is Python, so absolute costs include interpreter overhead and differ from native or hardware-accelerated implementations; MinIO ran as a single node without erasure coding; results describe one instance pair in one region. liboqs is intended for prototyping, not production. **Construct.** "Handshake overhead" is per-object key establishment, not a network handshake; transport security was held constant.

X. Conclusion

We built and validated an instrumented, reproducible benchmark of post-quantum envelope encryption for S3-compatible object storage and measured it from 1 to 500 concurrent workers and from 1 KiB to 500 MiB objects on two separate cloud instances. ML-KEM-768 adds 0.045–0.047 ms per object, is statistically equivalent to X25519 within ±5 % in 38 of 44 comparisons, with none confirmed to differ by more, while RSA-4096 limits small-object reads to 0.38–0.59 of ML-KEM's throughput. Bounded session amortization helps only when the client CPU is the bottleneck, and a queueing model with load-independent service time does not predict where response time collapses.

Artifact

Code, configurations, per-run summaries, the raw-data validator and the analysis scripts that generate every figure, table and number in this paper (`analysis/summarize.py`, `final_analysis.py`, `make_paper_assets.py`) are in the project repository, organized following ACM artifact guidelines [17]. The raw request records are archived separately.

References

[1] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," NIST, Federal Information Processing Standards Publication FIPS 203, Aug. 2024.

[2] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, 2010.

[3] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST, Tech. Rep. SP 800-38D, 2007.

[4] MinIO, Inc., "MinIO object storage server, release RELEASE.2025-10-15T17-29-55Z (repository archived april 2026)," https://github.com/minio/minio, 2025.

[5] Open Quantum Safe Project, "liboqs: C library for prototyping and experimenting with quantum-resistant cryptography, version 0.16.0," https://github.com/open-quantum-safe/liboqs, 2026.

[6] N. Rodriguez-Alvarez, "Performance and storage analysis of CRYSTALS-Kyber as a post quantum replacement for RSA and ECC," arXiv:2508.01694, 2025.

[7] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," RFC 8446, 2018.

[8] D. Connolly, P. Schwabe, and B. Westerbaan, "X-Wing: General-purpose hybrid post-quantum KEM," IETF Internet-Draft draft-connolly-cfrg-xwing-kem, 2024.

[9] L. F. Prikutse, R. E. Turkson, A.-S. Yussiff, A.-L. Yussiff, and M. G. Asante-Mensah, "A commitment-based hybrid post-quantum cryptographic model for multi-file cloud storage," arXiv:2608.13138, 2026.

[10] Amazon Web Services, "Reducing the cost of SSE-KMS with Amazon S3 Bucket Keys," Amazon S3 User Guide, https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-key.html, 2026.

[11] ——, "Data key caching details," AWS Encryption SDK Developer Guide, https://docs.aws.amazon.com/encryption-sdk/latest/developer-guide/data-caching-details.html, 2026.

[12] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," RFC 8017, 2016.

[13] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," RFC 7748, 2016.

[14] Open Quantum Safe Project, "liboqs-python: Python 3 bindings for liboqs, version 0.16.0.1," https://github.com/open-quantum-safe/liboqs-python, 2026.

[15] Python Cryptographic Authority, "cryptography, version 50.0.1," https://cryptography.io, 2026.

[16] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[17] Association for Computing Machinery, "Artifact review and badging, version 1.1," https://www.acm.org/publications/policies/artifact-review-and-badging-current, 2020.